

Arquitectura de Computadoras

## **Proyecto Rasterizador de Píxeles**

Comisión Número 19

Integrantes:

Duval Santino 152860,  
García Damiani Pedro 152029,  
Pelegri Carrin Juan Manuel 151852

*Versión de Logisim-Evolution: v4.1.0*

*El siguiente documento es el informe completo de las dos etapas que se llevaron a cabo el primer cuatrimestre del año 2026 en la materia Arquitectura de Computadoras para la realización de un circuito Rasterizador de píxeles.*

Arquitectura de Computadoras

Etapa N°1 del Proyecto

Fecha de Entrega: 21/04/2026

Comisión Número 19

Integrantes:

Duval Santino 152860,  
García Damiani Pedro 152029,  
Pelegrina Carrin Juan Manuel 151852

Versión de Logisim-Evolution: v4.1.0

**Consigna: “Etapa 1: Se desarrollará un sumador Carry Skip Adder de 16 bits implementado en el Trabajo práctico 3: TEMA A: Ejercicio 6 inciso a”.**

En esta etapa implementamos un Carry-Skip Adder de 16 bits utilizando cuatro sumadores ripple de 4 bits cada uno. Realizamos el diseño mediante el planteo de los subcircuitos necesarios para diseñar el Carry Skip Adder.

La jerarquía sería **Carry Skip Adder ← Ripple Adder ← Full Adder.**

### FULL ADDER

El primer subcircuito es el Full Adder, que dados los bits A y B y un Carry de entrada C\_in computa una suma y devuelve el bit S y un Carry de Salida C\_out.

La ecuación lógica para el sumador en álgebra booleana es la siguiente:

$$S = A \oplus B \oplus C_{in}$$

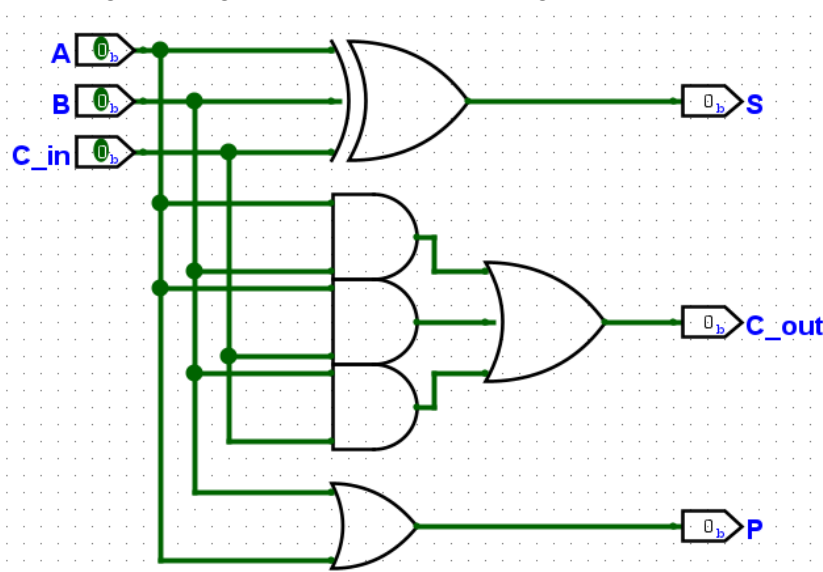
$$C_{out} = A \cdot B + P \cdot C_{in}$$

Donde C\_in es el bit de carry de entrada para calcular S, C\_out es el carry que devuelve la suma en la i-ésima posición y P es la propagación y es de la forma  $P = A \oplus B$ .

La tabla de verdad del circuito es la siguiente (donde C sería C\_in en el circuito):

| A | B | C | C <sub>OUT</sub> | S |
|---|---|---|------------------|---|
| 0 | 0 | 0 | 0                | 0 |
| 0 | 0 | 1 | 0                | 1 |
| 0 | 1 | 0 | 0                | 1 |
| 0 | 1 | 1 | 1                | 0 |
| 1 | 0 | 0 | 0                | 1 |
| 1 | 0 | 1 | 1                | 0 |
| 1 | 1 | 0 | 1                | 0 |
| 1 | 1 | 1 | 1                | 1 |

Y su diagrama lógico implementado en logisim con compuertas:



Este circuito cuenta con dos niveles de compuertas, 3 pines de entrada de 1 bit y 3 de salida de 1 bit .

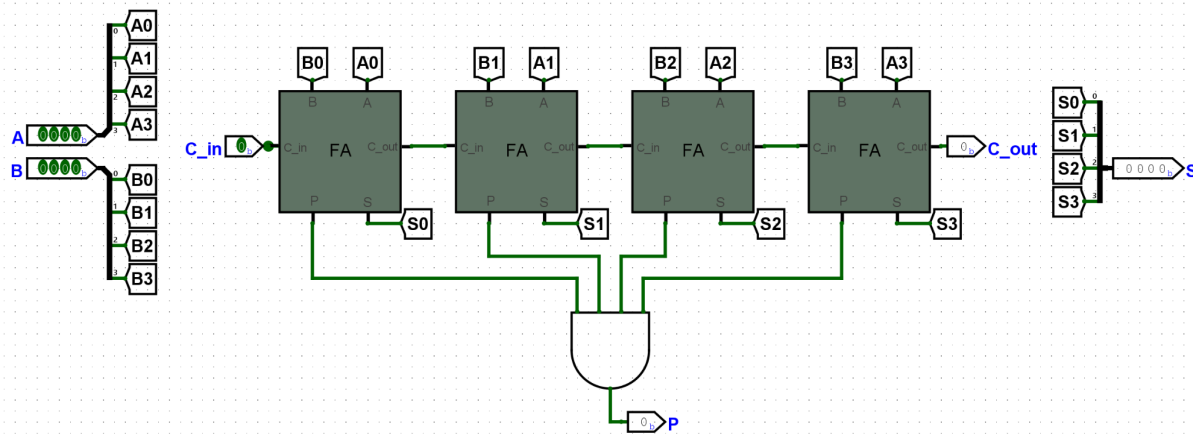
El Xor que calcula  $S = A \oplus B \oplus C_{in}$  tiene 3 pines de entrada de 1 bit y está configurado en su comportamiento con el modo de imparidad de variables, que simula el comportamiento de varios (en este caso 2) Xor's concatenados. Ésta es una optimización por sobre concatenar dos Xor's para conseguir el mismo resultado, pues un Xor de tres entradas computa su función ligeramente más lento que uno de dos, mientras que usar 2 de 2 introduce un nivel de compuertas innecesario al cálculo que duplica el tiempo de cómputo con respecto a un Xor de dos entradas.

## RIPPLE ADDER

Continuamos con los subcircuitos diseñando un Ripple Adder. Los Ripple Adders consiguen computar el resultado de una suma de  $n$  bits a través de  $n$  Full Adders conectados en cascada. Para esto, toma por entrada 2 números binarios de  $n$  bits y 1 bit como Carry de entrada y retorna 1 número binario de  $n$  bits y 1 bit como Carry de salida.

Los datos de los bits a los cuales los Full Adders han de computar su suma son recibidos por estos como inputs al Ripple Adder. Una ventaja de esta manera de introducir los datos es que las propagaciones de todos los Full Adders pueden ser calculadas de manera sincrónica en el tiempo que tarda un nivel de compuertas.

El diagrama lógico que representa nuestro diseño de un Ripple Adder de 4 bits es el siguiente:



Las entradas A y B del Adder son pines de 4 bits cada una, de modo de poder sumar los 4 bits requeridos por las especificaciones de la consigna. Éstas entradas luego son divididas en sus 4 valores y transportadas con Tunnels a las entradas de los Full Adders para procesar las sumas correspondientes.

La limitación infranqueable del diseño es el hecho de que para yo poder calcular la suma Si requiero primero el bit de carry Ci que debe ser calculado en un Full Adder previo (a menos que sea el primer Full Adder) y por lo tanto Si no se puede computar de forma sincrónica.

Si el retardo de una compuerta lógica es  $t$ , entonces  $2t$  es el retardo de un Full Adder, y por lo tanto, este Ripple Adder de 4 bits tiene un retardo de  $2t \cdot 4 = 8t$ .

El pin de entrada  $C_{in}$  ingresa un carry inicial para comenzar el proceso. Luego, los cuatro resultados  $S_0$ ,  $S_1$ ,  $S_2$  y  $S_3$  de las sumas son combinados para devolver una salida de 4 bits de nombre  $S$  y  $C_{out}$ .

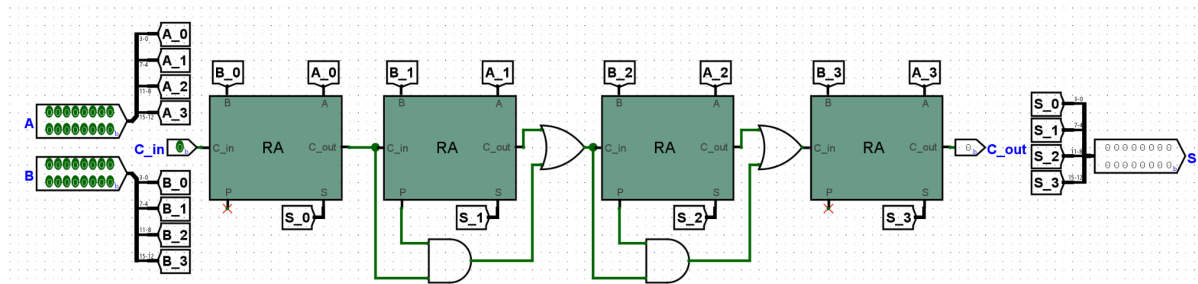
Por último, este Ripple Adder calcula la propagación del Carry de entrada y lo retorna mediante la salida  $P = (A_0+B_0) \cdot (A_1+B_1) \cdot (A_2+B_2) \cdot (A_3+B_3)$

### CARRY SKIP ADDER

El circuito principal a diseñar para la etapa primera del proyecto es el Carry Skip Adder. Este circuito se construye al conectar en cascada varios Ripple Adders y logra optimizar el retardo de hacer sumas al utilizar la propagación combinada con el carry de entrada de cada Ripple Adder para poder predecir si este va a tener un carry de salida, así pudiendo comenzar el procedimiento de suma del próximo Ripple Adder sin requerir que todas las sumas dentro de este Ripple Adder sea terminada de antemano.

*Si se tiene una propagación del carry a lo largo de todo un RA (Ripple Adder) y además ese RA tuvo un carry de entrada puedo estar seguro de que, independientemente del resultado del cálculo, va a haber un carry de salida.*

La implementación del diseño en forma de diagrama lógico:



La entrada  $C_{in}$  representa el carry de entrada y las entradas  $A$  y  $B$  de 16 bits representan los dos números que van a ser sumados y son divididos en 4 buses de 4 bits cada uno para ser ingresados en los RA de 4 bits correspondientes.

sea  $\Delta G$  el retardo de una compuerta lógica,  $k$  la cantidad de bits de los operandos que se suman en ripple y  $n$  la cantidad de bits por operando a sumar

El tiempo de suma  $T$  de un Carry Skip Adder está dado por  $\Delta G$

$T = t_{\text{ripple en } k \text{ bits}} + (n/k - 2)(t_{\text{AND}} + t_{\text{OR}}) + t_{\text{ripple en } k \text{ bits}}$

$T = \Delta G (4k + 2n/k - 4)$

En este caso en particular nuestro ripples son de  $k = 4$  bits y nuestra cantidad de bits totales es de  $n = 16$  bits. luego  $T = \Delta G(16+8-4) = 20 \Delta G$ . Además el T de tener simplemente RA's sin el Carry Skip Adder sería de  $4*2*4 = 32 \Delta G$ . O sea que este diseño plantea una gran optimización en prácticamente todos los casos menos el peor de todos.

Por último los resultados de cada RA son combinados y salen por el pin S, además del C\_out.

Este circuito particularmente no está diseñado para ser concatenado con otros del mismo tipo sin perder una parte de su eficiencia en el camino, pues la comparación del carry de entrada y propagación del último RA es saltada en pos de ahorrar en compuertas lógicas.

## CONCLUSIÓN

Se implementó correctamente un Carry-Skip Adder de 16 bits en Logisim-Evolution utilizando Full Adders y bloques Ripple Adder de 4 bits. La lógica de skip permitió mejorar el tiempo de propagación del carry respecto de un Ripple Adder tradicional. Las pruebas realizadas mostraron resultados correctos.

### Registro de Correcciones Etapa N°1:

\*Debería haber consistencia en cuanto a la persona gramatical utilizada, usando siempre impersonal o 1º persona del plural. (ej. no combinar "se implementó", "yo tengo" y "nuestros ripples").

-Correcciones varias en el texto.

\*Sería bueno agregar una introducción, que hable del objetivo general del proyecto, más allá de la etapa 1.

\*El sumador no solo suma números no signados. También puede sumar números en complemento a la base.

*Continuamos con los subcircuitos diseñando un Ripple Adder. Los Ripple Adders consiguen computar el resultado de una suma de  $n$  bits a través de  $n$  Full Adders conectados en cascada. Para esto, toma por entrada 2 números binarios **no signados** de  $n$  bits y 1 bit como Carry de entrada y retorna 1 número binario **no signado** de  $n$  bits y 1 bit como Carry de salida.*

-Ambas menciones de "no signado" quitadas.

Arquitectura de Computadoras

Etapa N°2 del Proyecto

Fecha de Entrega: 12/05/2026

Comisión Número 19

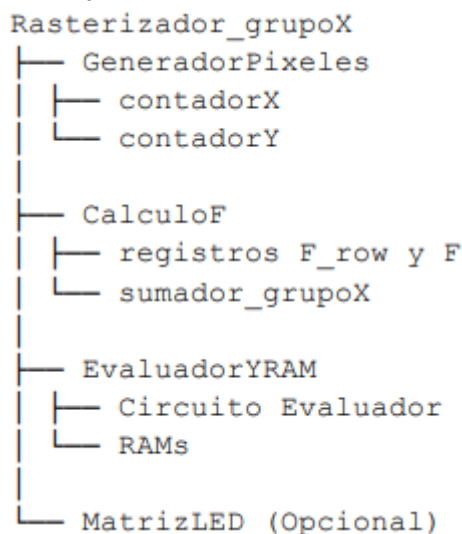
Integrantes:

Duval Santino 152860,  
García Damiani Pedro 152029,  
Pelegri Carrin Juan Manuel 151852

Versión de Logisim-Evolution: v4.1.0

**Consigna:** “Se desarrollará el rasterizador. En esta etapa se implementarán los circuitos necesarios para la rasterización completa, integrando los contadores para la generación de coordenadas, los registros intermedios, la lógica de control y el evaluador final. Esto permitirá procesar las coordenadas y evaluar la función lineal para determinar si cada píxel debe dibujarse”.

Para esta etapa tuvimos que desarrollar un rasterizador de píxeles para poder evaluar para píxeles dentro de una matriz de 16x16 píxeles si cumple con los lineamientos de funciones lineales de la forma  $Ax + By + C \geq 0$  siguiendo la estructura planteada por el enunciado del proyecto:



## CONTADOR X

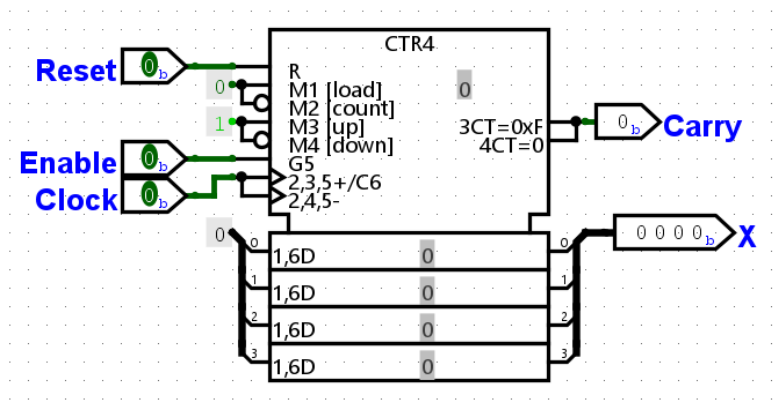
El **contadorX** es uno de los dos subcircuitos básicos que componen el subcircuito **GeneradorPíxeles** y, dadas las entradas **Reset**, **Enable** y **Clock** (las tres de 1 bit), siempre que esté habilitado (bit **Enable** es 1), para cada ciclo del reloj **Clock** devuelve a través de su salida **X** de 4 bits un número  $X_{i+1} = X_i + 1$ , donde  $X_i$  está inicializado como 0. Además, una vez completado un ciclo completo de X, donde X es 15, X regresa a 0 y el bit **Carry** de salida envía una señal con un 1.

De forma resumida y simplificada podríamos decir que el **contadorX** al recibir un ciclo del reloj suma uno al valor que soltó de manera previa.

Todo este comportamiento es logrado a través de un circuito provisto por logisim de nombre “Counter” configurado para 4 bits de datos, con el resto de su configuración estándar.

El diagrama lógico de la implementación del contador:





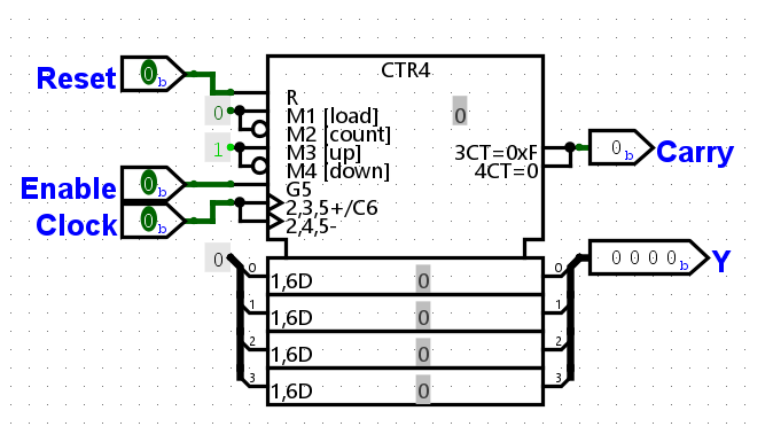
Las entradas y salidas ya discutidas se pueden mapear de forma directa con las entradas y las salidas del contador, donde **Reset** entra a R, **Enable** a G5 y **Clock** a la entrada para el reloj del counter, además de **Carry** recibiendo las señales que suelta el counter para el valor 15 y **X** recibiendo el número actual que está contando el counter.

Por último, además de las entradas que son pines, completamos el resto de entradas que tiene el counter con constantes, de modo que M2 se activa para contar, M3 se activa para que la cuenta se haga hacia arriba y en Data hay un 0 que no se utiliza, pues M1 está desactivado.

## CONTADOR Y

El **contadorY** es casi exactamente el mismo circuito que **contadorX**, con la excepción de que el nombre de la salida para los números contados es de nombre **Y** en vez de **X**, luego, toda la explicación de **contadorX** es válida para **contadorY** si se piensa en términos de **Y**.

El diagrama lógico de **contadorY**, casi idéntico al de **contadorX**:

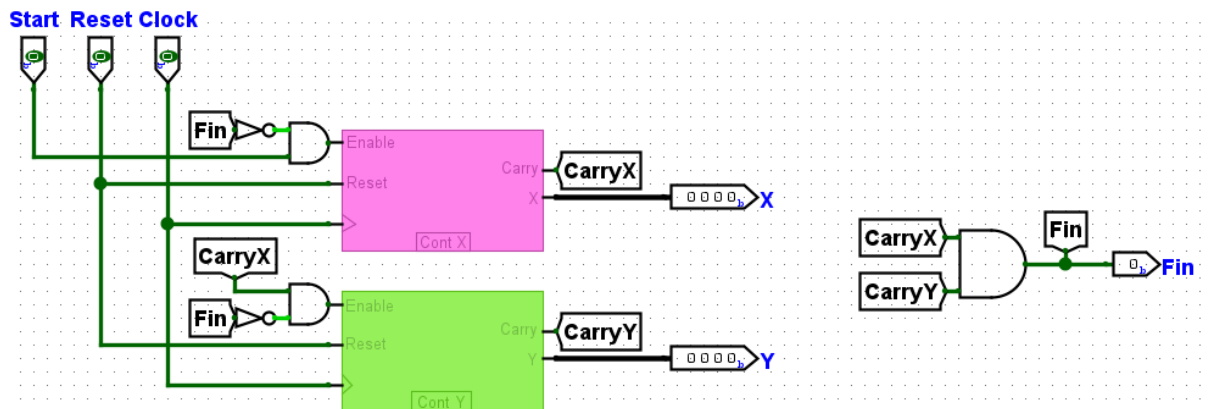


## GENERADOR PÍXELES

Haciendo uso de los contadores X e Y, **GeneradorPixeles** toma las entradas de **Start**, **Reset** y **Clock** y simula un recorrido horizontal de un sistema de coordenadas donde **X** es el eje horizontal e **Y** es el eje vertical.

Una vez **contadorX** llega a su carry, usando esa señal se puede habilitar el **contadorY** por un ciclo.. Si interpretamos las coordenadas de **X** y de **Y** como una terna de la forma (X,Y) el inicio del recorrido se encuentra en (0,0), y **X** va aumentando para cada ciclo del reloj hasta (15,0) que sería el límite de la matriz. En el próximo ciclo, como **X** se reinicia y **Y** aumenta, entra a la coordenada (0,1) y vuelve a recorrer **X**. Este ciclo se repite hasta que llega a la coordenada (15,15), donde el próximo carry de Y envía la señal **Fin**, indicando que ya se recorrió toda la matriz de 16x16.

A continuación el diagrama lógico que permite este comportamiento:



Veamos inicialmente las funcionalidades que se aplican a las señales de entrada.

- **Start** funciona como el **Enable** de este circuito. Siempre que esa señal se encuentre en 1 los contadores podrán hacer uso de los ciclos del reloj que les lleguen, siempre que la otra condición del **Enable** se valide eso es. Para **contadorX** además de que el circuito se encuentre en **Start**, no debe ser el **Fin** y para **contadorY** solo usa ciclos del reloj cuando no es el **Fin**, pero además solo en las ocasiones en las que **X** pasó de 15.
- **Reset** es simplemente un switch de reset para los contadores, que devuelve los valores a (0,0) cuando es 1.
- **Clock** es el reloj que activa los contadores y hace uso de su lógica.
- Por último las salidas del **GeneradorPixeles** son las coordenadas **X** e **Y** actuales y el valor de **Fin**, que está determinado por si existe **X** e **Y** se encuentran en el valor 15.

## CALCULO F

Para la explicación del cálculo de F vamos a hacer una excepción en cuanto a la explicación de sus subcircuitos según la estructura y vamos a ir directamente al funcionamiento del circuito en conjunto, pues los registros son provistos por logisim directamente como deben usarse y es el caso que el sumador de 16 bits es el mismo que ya abordamos extensamente en la Etapa N°1 del proyecto.

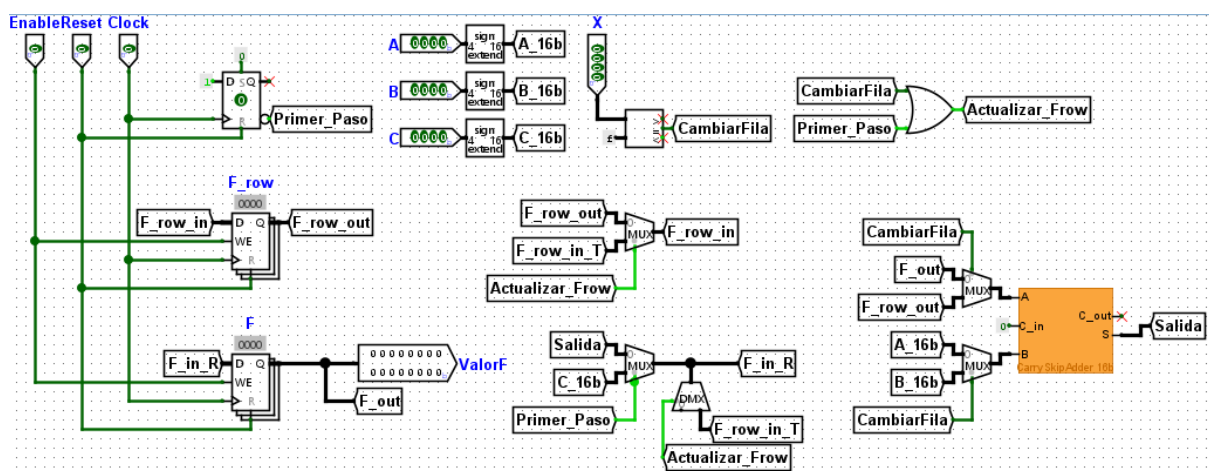
Este subcircuito contiene toda la lógica que permite calcular el valor de  $Ax + By + C$  o sea, calcula F.

**CalculoF** toma las coordenadas (X,Y) actuales para un ciclo del reloj determinado y devuelve F en ese mismo ciclo. Además consigue hacer estos cálculos de una manera muy eficiente, no requiriendo el uso de circuitos multiplicadores para hacer los cálculos, reemplazando este comportamiento caro por el uso de registros.

De manera simplificada **CalculoF** toma **A**, un número en complemento a dos de 4 bits, y para cada coordenada de **X** dentro del 16x16 lo suma al registro que guarda los valores de **F**. Una vez que salta de fila puedo entonces sumar un nuevo valor de **B**, que tiene las mismas restricciones de **A**, al mismo registro, una vez por salto de **Y**, además de guardar este valor para usarlo en sumas subsecuentes.

El diagrama lógico de **CalculoF** es, además de complejo, extenso, y conviene irlo explicando paso a paso.

Aún así presentamos directamente el diagrama:



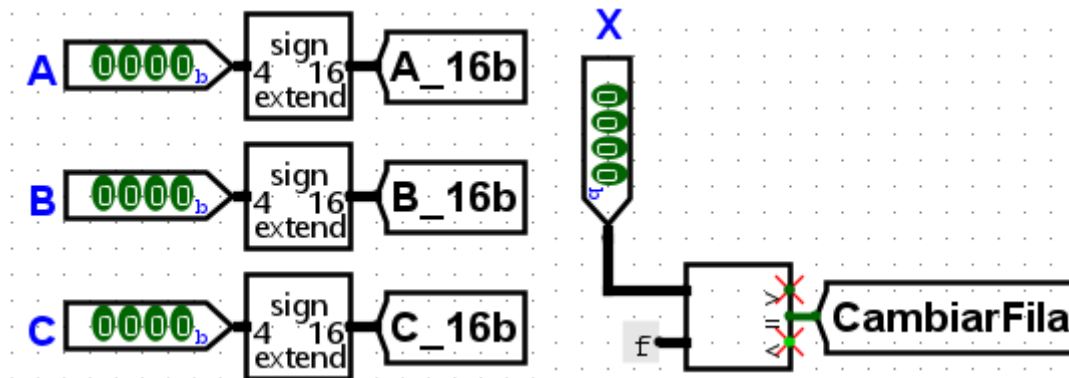
Comenzando por las entradas, **CalculoF** recibe su **Enable**, **Reset** y su **Clock**. Los usos de estas entradas son puramente de control.

- El **Enable** cumple el propósito de habilitar el uso de los registros **F\_row** y **F**, ingresando a sus entradas **WE**, que les permiten procesar los ciclos del reloj.
- **Reset** resetea el D flip-flop encargado de discernir si el cálculo está en su primer paso y los registros cuando vale 1.
- Por último **Clock** manda señales del reloj a los registros y al D flip-flop.

Las entradas de datos para el cálculo de datos de F son los números en complemento a dos de 4 bits **A**, **B** y **C**, además del valor de 4 bits de **X** que se usa para saber si hay carry **X** (la

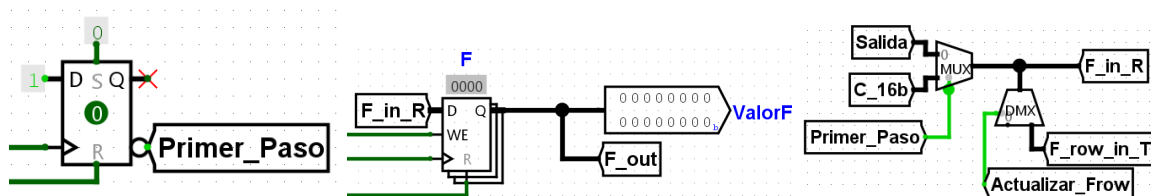
razón de ésta comparación que puede llegar a parecer de más es porque la especificación de **GeneradorPíxeles** afirma que las salidas de ese circuito pueden ser únicamente X e Y: “Los valores de salida del circuito serán X e Y...”).

Para poder ser sumados de forma efectiva **A**, **B** y **C** son extendidos a 16 bits con el componente Bit extender configurado de 4→16 y con extensión de signo. **X** se compara con 15 para saber si habría carry en el **contadorX**.

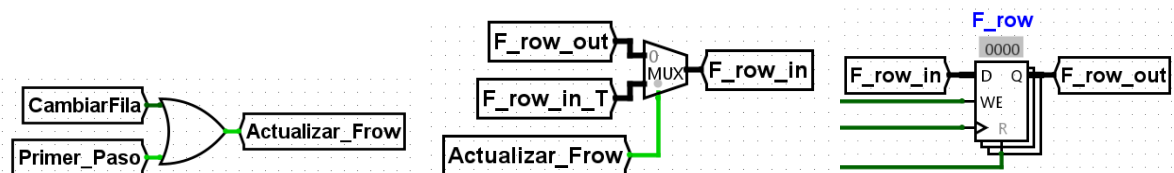


El primer paso que toma **CalculoF** es el de saber si tengo que agregar que siempre y para todos los valores posibles de F está presente. Esto implica que C debe ser sumado una única vez a F y su valor no debe ser olvidado. Por ejemplo, para (0,0), F es C.

Con esa finalidad el circuito cuenta con un D Flip-Flop que solo emite una señal 1 por su salida Q' en la única ocasión de que el clock no haya hecho ni un solo ciclo.



Cuando estoy en el primer paso la salida que entra al registro **F** no es la del sumador, si no **C**. Una vez el reloj pasa un ciclo ese valor de **C** se carga en **F** y escapa directamente por **ValorF**. **C** también se carga en **F\_row** para poder saber cuánto se le debe sumar al F inicial de cada fila.



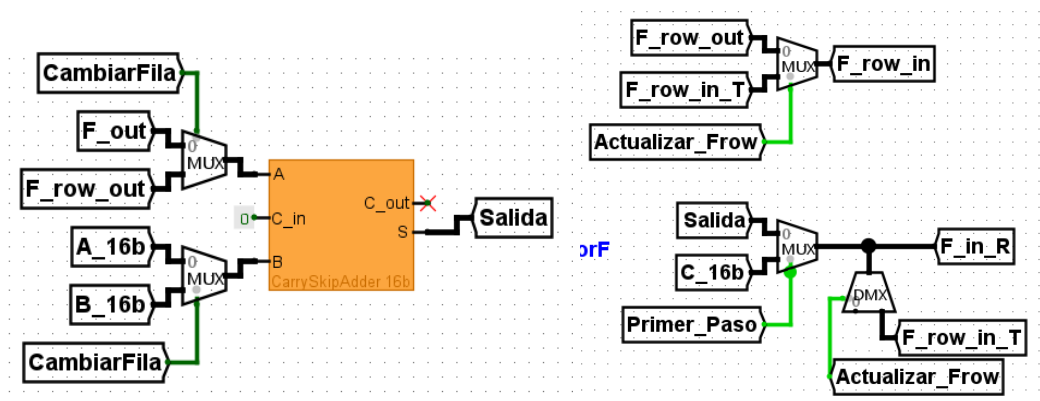
El DMX pasa el valor de la salida del cálculo del sumador, o el valor de **C**, a **F\_row** además de a **F** solo en el caso en el que haya que actualizarlo.

A partir de los próximos ciclos ya comienza a recibir valores de **X** y de **Y** que cambian. El algoritmo de **CalculoF** es el siguiente:

- Chequea si se encuentra en el primer paso, caso positivo carga **C** en **F** y en **F\_row**.
- Actualiza **F** sumándole **A** para cada desplazamiento de **X**.
- Cuando **X** alcanza su fin le carga a **F** y a **F\_row** el valor de **F\_row + B**.
- Cicla esos dos pasos hasta completar el recorrido.

Para sumarle **A** a **F** chequea que no haya que **CambiarFila** y entonces le suma a **F + A** y lo saca por Salida. Salida luego entra a **F**.

En caso de **CambiarFila** le suma a **F\_row + B** y manda ese valor a **F** y a **F\_row** (pues se activa **Actualizar\_Frow**).



El MUX que ingresa los valores a **F\_row** también le ingresa su propio valor de salida a **F\_row** para evitar que se reinicie.

El **C\_in** de del **CarrySkipAdder** es un 0 constante y el **C\_out** no nos interesa.

## **EVALUADOR Y RAM**

Este circuito se encarga de almacenar los valores calculados de **F** y determinar si cada píxel de la matriz debe dibujarse o no, utilizando memorias RAM, junto con lógica de control para seleccionar qué datos ingresarles.

Las señales **Enable**, **Reset** y **Clock** controlan el funcionamiento secuencial del sistema:

- **Enable** habilita las operaciones de lectura y escritura.
- **Reset** reinicia el contenido de las memorias.
- **Clock** sincroniza las escrituras en ambas RAM.

Ante las entradas **X** e **Y** de 4 bits, el circuito las combina mediante un splitter para generar un bus **Address** de 8 bits. Esto permite que cada par (x, y) quede asociado a una posición única dentro de las memorias.

El diseño cuenta con dos memorias principales:

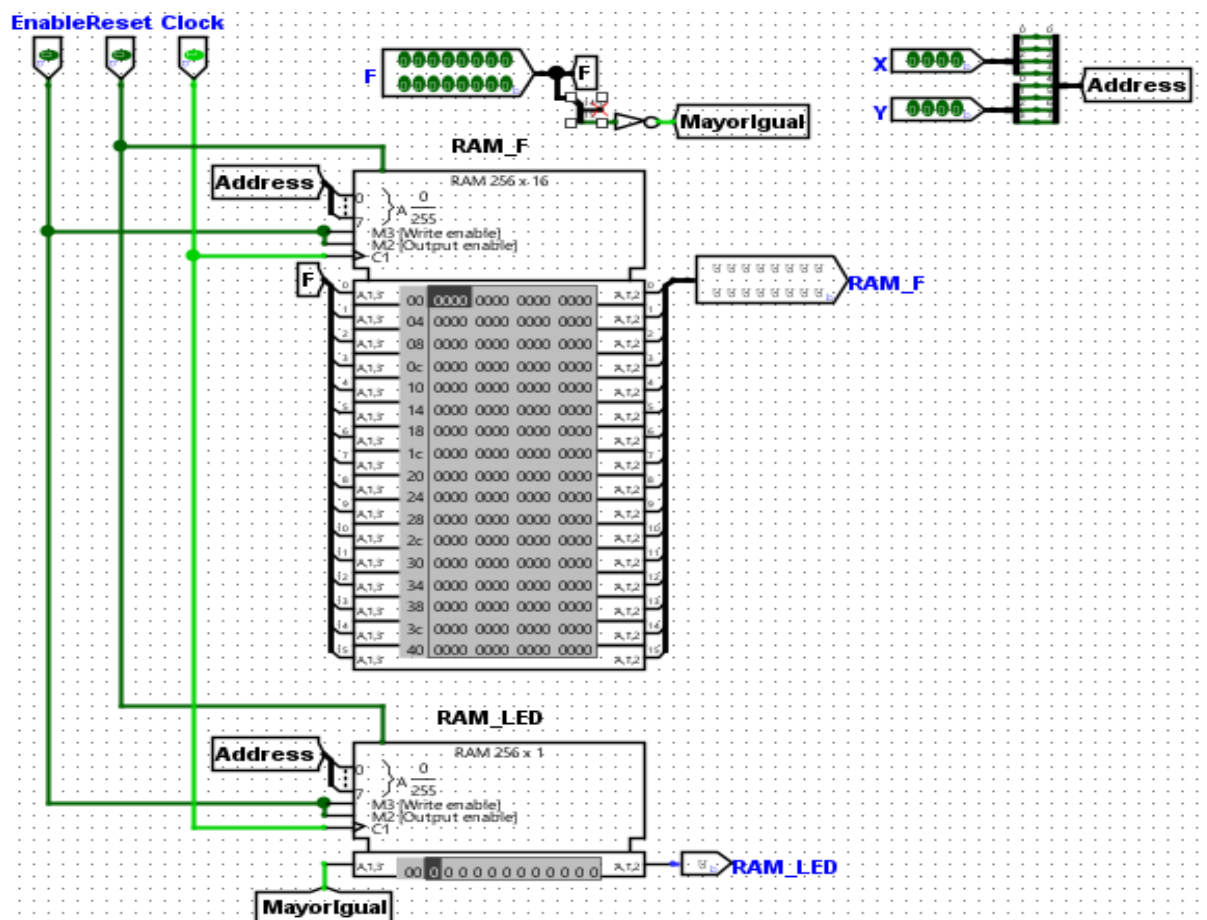
- **RAM\_F** 256x16 donde se almacena el valor completo de **F(x, y)**
- **RAM\_LED** 256x1 donde se almacena el resultado de la evaluación lógica del píxel.

La memoria **RAM\_F** recibe directamente el valor de la entrada **F** y lo guarda en la dirección indicada por **Address**.

Para verificar la condición  $F(x, y) \geq 0$ , el circuito utiliza un splitter que extrae únicamente el bit más significativo (**MSB**) de **F**, el cual representa el signo.

- Si **MSB**=0 (positivo o cero), la compuerta NOT lo convierte en 1, marcando el píxel como válido para dibujarse.
- Si **MSB**=1 (negativo), la compuerta NOT lo convierte en 0, marcando el píxel como invalido.

Este bit resultante se guarda en **RAM\_LED** en la dirección indicada por **Address**, almacenando así el estado lógico del píxel correspondiente.



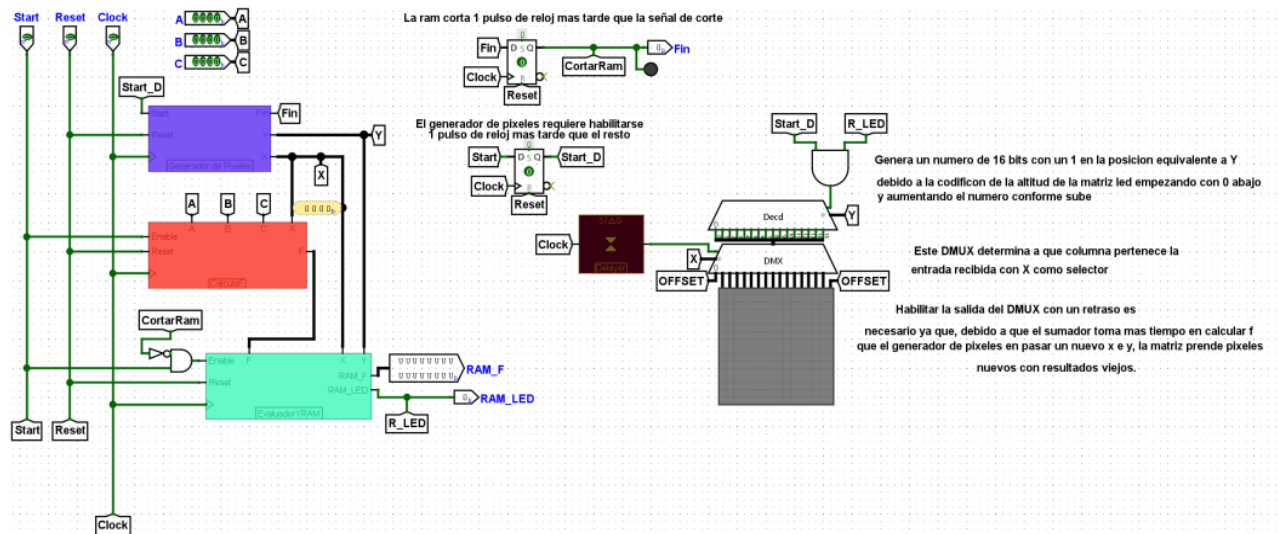
## RASTERIZADOR

Este último circuito es la culminación del proyecto y combina todos los subcircuitos previamente explicados para poder devolver las señales indicadas por la consigna.

El **GeneradorPíxeles** recorre la matriz de 16x16, **CalculoF** calcula **F** para cada par de coordenadas y **EvaludadorYRAM** confirma, para cada  $F(X, Y) \geq 0$ . Además, como añadido

extra que tiene el circuito está presente un subcircuito opcional que le da una salida gráfica al circuito mediante una matriz led.

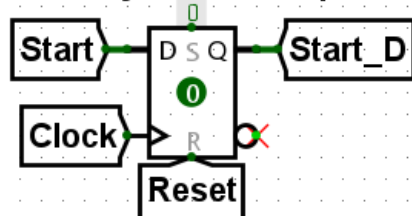
El diagrama lógico del circuito completo:



Comencemos por última vez explicando las entradas **Start**, **Reset** y **Clock**, que son universales para los subcircuitos y tienen su origen en este.

- **Start** inicia de manera directa **CalculoF** y **EvaluadorYRAM**. Después tiene su salida retrasada por un pulso del reloj para habilitar el **GeneradorPixeles**. La función de esto es poder cargar  $F(0,0) = C$  de una antes de empezar a ciclar las coordenadas.

El generador de pixeles requiere habilitarse 1 pulso de reloj mas tarde que el resto

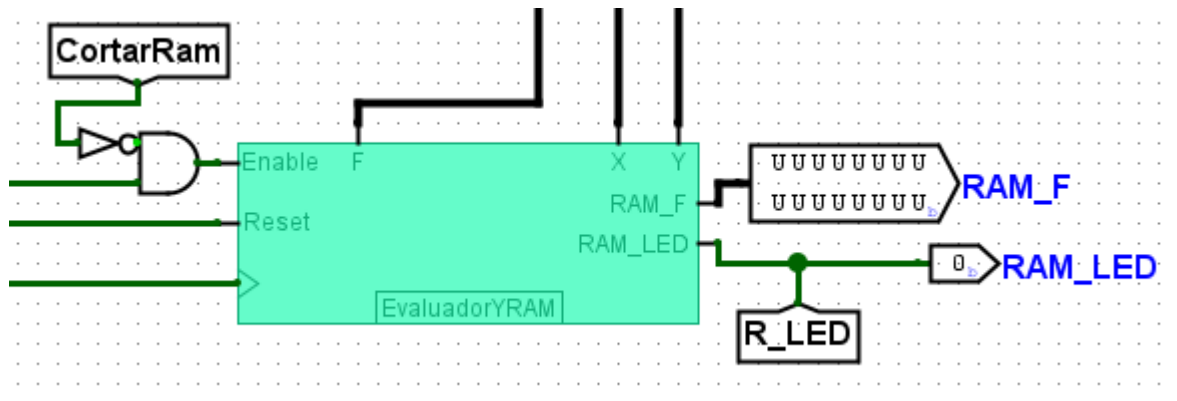


- **Reset** no cuenta con lógica extra aplicada a su señal. Hace lo que el nombre dice: cuando es 1 resetea todo. Lo único que no resetea es la matriz LED, la única manera real de resetearla es resetear la simulación por completo o esperar 1000 ticks (configuración personal del circuito).
- **Clock** tiene una política estricta de usar la señal como viene, solo en rising-edge. La única excepción es cuando le aplicamos un delay a su señal para matriz LED, que vuelve a ser una excepción a la mayoría de normas formales que sigue todo el circuito debido a su carácter opcional.

Las entradas de 4 bits en complemento a dos **A**, **B** y **C** entran directamente al **CalculoF**, el cual las usa para poder calcular el valor de  $F(X,Y)$  que va ciclando **GeneradorPixeles**. Esas **X** e **Y** que recorre el circuito también son enviadas a **EvaluadorYRAM** en conjunto con **F** y las salidas **RAM\_F** y **RAM\_LED** son salidas del rasterizador para la consigna. **RAM\_LED**

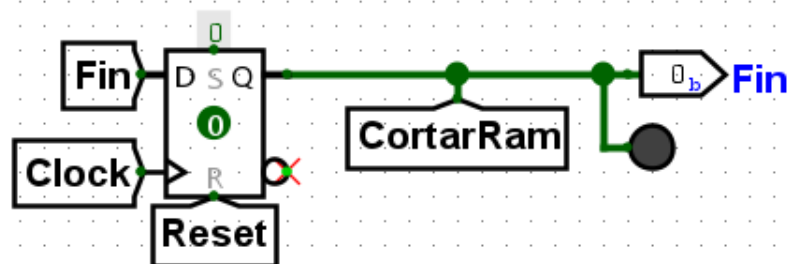


además distribuye su salida hacia el mecanismo de **Matriz\_LED** para poder procesar las salidas y mostrarlas gráficamente.



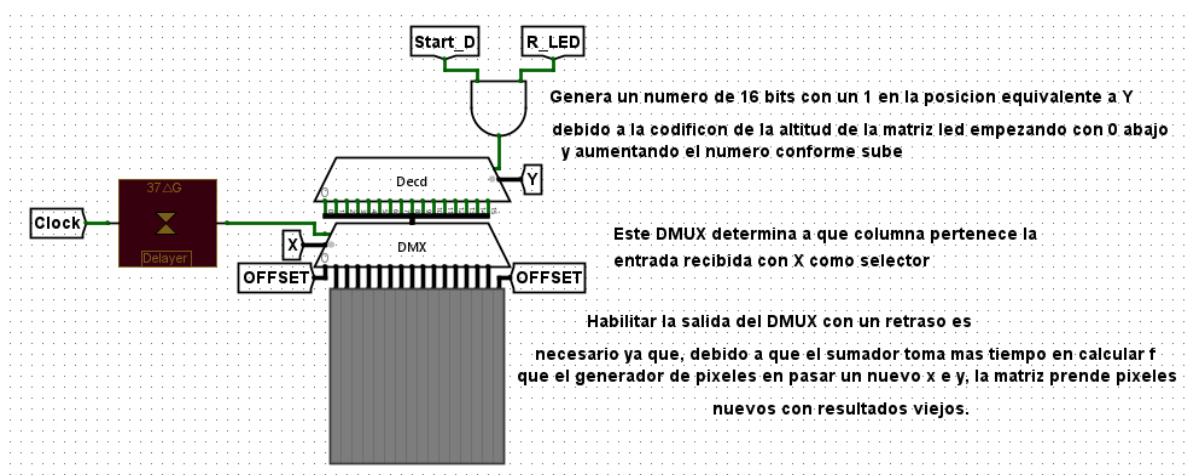
Además destacamos la condición **CortarRam**, que deshabilita **EvaluadorYRAM** según lo indicado por su mecanismo:

**La ram corta 1 pulso de reloj mas tarde que la señal de corte**



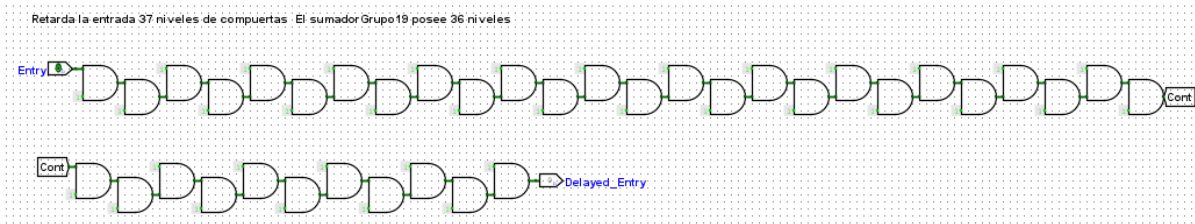
Fin es la última salida del **Rasterizador** y es una señal que da **GeneradorPixeles**.

Por último está el diagrama lógico de **Matriz\_LED**, con explicaciones:



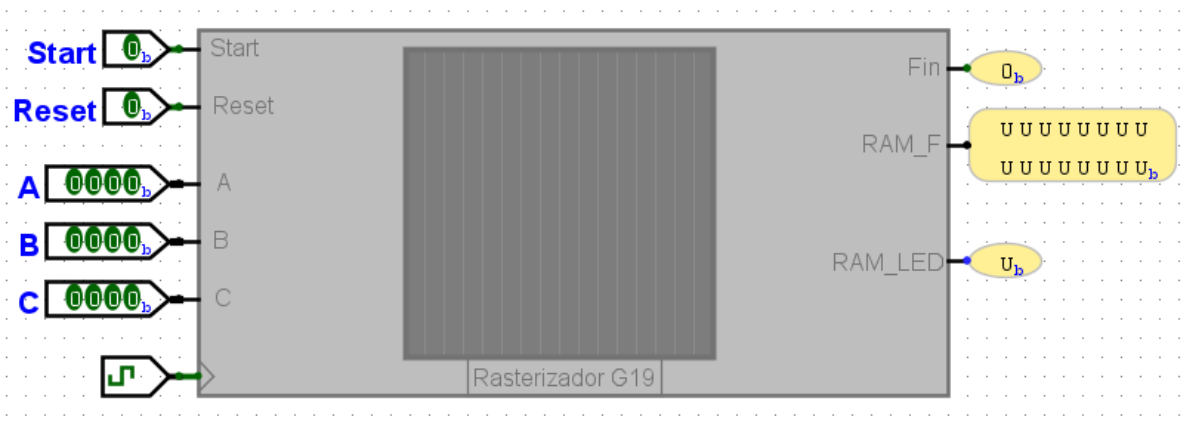
Y el **Delayer**:





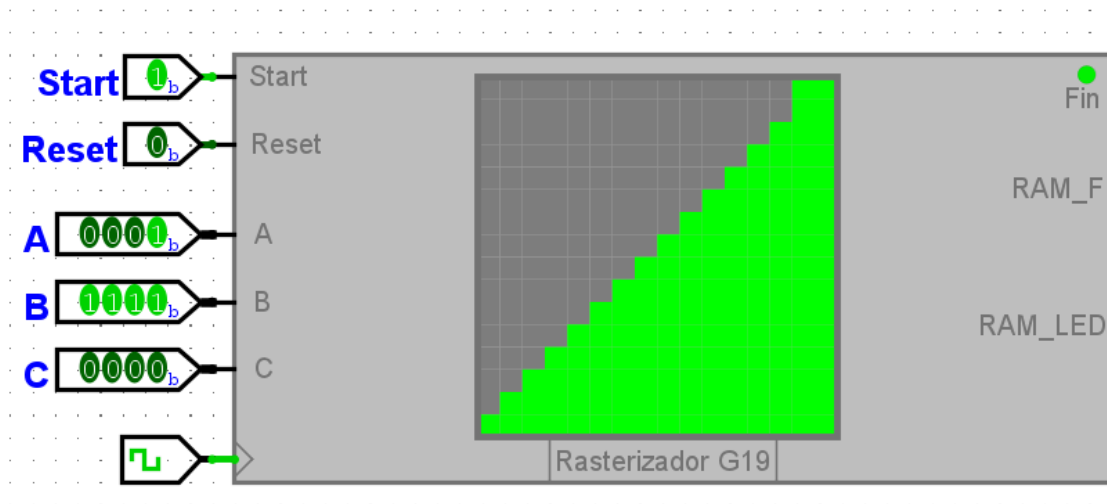
El cual, como explica la imagen, retrasa una entrada 37 niveles de compuerta, lo cual es útil por que permite que la **MatrizLed** espere a que el **CalculoF** retorne el valor de la coordenada actual.

Para comodidad de testeo el rasterizador incluye un circuito extra que no es parte de la especificación de nombre "Tester", que cuenta con el único clock real del circuito. Su exactitud no está asegurada y queda a discreción del usuario.



Ejemplos:

$$F = X - Y$$



### Configuracion

| Parámetro A | Parámetro B | Parámetro C |
|-------------|-------------|-------------|
| 1           | -1          | 0           |

Orden de memoria

- ☒ Row major (YX) solicitado
- ☐ Column major (XY)

Archivo de memoria RAM\_F (Logisim) ⓘ

  
**RAM\_F**  
Formato: v3.0 hex words plain — valores de 16 bits

Archivo de memoria RAM\_LED (Logisim) ⓘ

  
**RAM\_LED**  
Formato: v3.0 hex words plain — valores de 1 bit (0 o 1)

 Verificar Rasterizador

### Resultado Global

 **Todos los chequeos pasaron**  
El rasterizador funciona correctamente

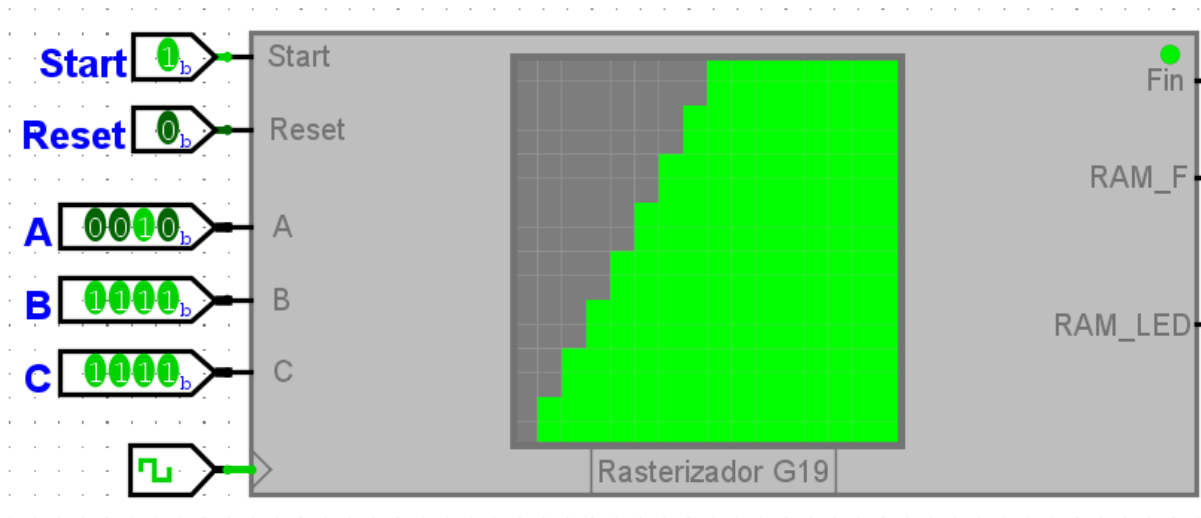
Funcion:  $f(X,Y) = X - Y$

Calculo de valores  
**100.0%**

Consistencia de RAMs  
**100.0%**

Coincidencia de graficos  
**100.0%**

$$F = 2X - Y - 1$$



### Configuracion

| Parámetro A | Parámetro B | Parámetro C |
|-------------|-------------|-------------|
| 2           | -1          | -1          |

Orden de memoria

☒ Row major (YX) solicitado ☐ Column major (XY)

Archivo de memoria RAM\_F (Logisim) 📎

↑

RAM\_F

Formato: v3.0 hex words plain — valores de 16 bits

Archivo de memoria RAM\_LED (Logisim) 📎

↑

RAM\_LED

Formato: v3.0 hex words plain — valores de 1 bit (0 o 1)

▶ Verificar Rasterizador

### Resultado Global

✓ **Todos los chequeos pasaron**  
El rasterizador funciona correctamente

Funcion:  $f(X,Y) = 2X - Y - 1$

Calculo de valores  
**100.0%**

Consistencia de RAMs  
**100.0%**

Coincidencia de graficos  
**100.0%**